Week 9 — Filesystems and Virtual Devices

Format: seven hands-on-first daily lessons collected into one mastery package

Suggested workload: approximately 21 focused hours, adjustable to mastery

> Begin with a prediction, run the canonical artifact, identify the mutable owner, and stop every claim at the boundary of the evidence.

This package studies the existing `projects/toyos` Week 9 milestone. Every challenge is an executable wrapper around that canonical project. The wrappers do not copy, patch, or simulate the kernel. Their recurring structure is:

```
purpose → prediction → evidence → explanation → limits
```

Week 9 adds two deliberately small vertical slices. The filesystem slice has two static ramfs inodes, four global open-file slots, offsets, absolute-path lookup, and ring-3 `open`, `read`, and `write`. The device slice enumerates PCI bus 0 with x86 configuration mechanism #1, identifies QEMU EDU `1234:11e8`, maps BAR0, checks the EDU identification register, and runs its liveness inversion operation. ToyOS has no real block layer, page cache, journaling, directories, close, permissions, IRQ-driven EDU operation, or DMA.

Daily roadmap

Day 1 — Why VFS objects exist

Start by inventorying `inode`, `file`, path, and descriptor state in the running kernel. Then distinguish identity, per-open position, naming, lookup, and ownership.

- Lesson: day-001-vfs-objects-and-ownership.md
- Challenge: `challenges/day-001-vfs-objects.sh`

Day 2 — Tiny ramfs operations

Trace absolute-path lookup, initialization, open-slot allocation, EOF, capacity checking, and offset movement in the two-inode ramfs.

- Lesson: day-002-ramfs-lookup-and-io.md
- Challenge: `challenges/day-002-ramfs-io.sh`

Day 3 — User descriptors and lifetimes

Follow user registers through bounded path copying and descriptor dispatch. Classify `-2`, `-9`, `-14`, `-22`, `-24`, and `-28` by detection boundary.

- Lesson: day-003-user-fd-syscalls.md
- Challenge: `challenges/day-003-user-fd.sh`

Day 4 — The missing storage stack

Use the implemented ramfs as a baseline, then build a conceptual file→page-cache→filesystem→block path without attributing any of those absent layers to ToyOS.

- Lesson: day-004-storage-stack-boundary.md
- Challenge: `challenges/day-004-storage-boundary.sh`

Day 5 — PCI discovery and BARs

Inspect configuration mechanism #1, bus/device/function addressing, vendor and device IDs, BAR decoding, and the assumptions behind finding QEMU EDU.

- Lesson: day-005-pci-enumeration-and-bars.md
- Challenge: `challenges/day-005-pci-edu.sh`

Day 6 — MMIO and the device contract

Trace mapped register access and the EDU liveness operation. Separate compiler visibility, CPU ordering, device semantics, and kernel policy; mark IRQ and DMA as explicit future boundaries.

- Lesson: day-006-mmio-ordering-and-edu.md
- Challenge: `challenges/day-006-mmio-edu.sh`

Day 7 — Integrated fault/debug report

Generate one report spanning ring-3 open/read/write, VFS/ramfs ownership, PCI/MMIO setup, both controlled rejection profiles, ToyOS/xv6/Linux comparisons, and the next driver path.

- Lesson: day-007-integrated-filesystem-device-report.md
- Challenge: `challenges/day-007-week9-report.sh`

Integrated checkpoint

Narrate both chains without merging their owners:

```
user path bytes → checked copy → VFS lookup → inode identity
→ open-file slot/offset → fd → checked user buffer → ramfs bytes

PCI config ports → vendor/device match → BAR decode → physical MMIO page
→ kernel mapping → volatile register access → EDU-defined inversion
```

For every arrow state:

1. the architecture or device contract;
2. the kernel policy;
3. the mutable owner and lifetime;
4. the error boundary;
5. the exact source, ELF, or checked QEMU evidence.

Architecture scope

The canonical implementation is x86-64. Its PCI configuration access uses I/O-port instructions, and its mapper invalidates a changed translation with `invlpg`. AArch64 systems commonly discover devices through firmware/device tree or ACPI, map MMIO with device memory attributes, and use `dmb`/`dsb` as required. RISC-V platforms commonly describe devices in a device tree, use memory-mapped host bridges, and use `fence` where ordering requires it. Those differences matter at the device-access boundary; the VFS policy concepts do not depend on one ISA.

Debugging discipline

Establish artifact identity before interpreting a fault. The normal Week 9 profile requires QEMU's `edu` device, host status 33, and marker `WEEK9: PASS`. The filesystem rejection profile requires status 47 and `EXPECTED-FAULT: filesystem PASS`. The invalid-register profile requires status 49 and `EXPECTED-FAULT: device PASS`. These are checked negative tests, not accidental crashes.

The canonical `gdb/week9.gdb` stops at `kernel_main`, `vfs_init`, `edu_init`, and `syscall_dispatch`. Suggested observations are predictions until actually captured. For a filesystem failure inspect the copied path, selected inode, file slot, offset, requested length, and return value. For a device failure inspect the PCI address/data transaction, raw BAR, masked physical address, PTE, register offset, width, value, and exact QEMU command line.

Reference discipline

xv6-riscv comparison landmarks commonly include `kernel/file.c`, `kernel/fs.c`, `kernel/bio.c`, `kernel/log.c`, `kernel/sysfile.c`, and its virtio disk driver; verify the revision used. Linux landmarks include `fs/`, `mm/filemap.c`, `block/`, `drivers/pci/`, `include/linux/io.h`, and a concrete driver under `drivers/`; inspect a current tree before line-level claims.

Do not auto-fill mastery, confidence, hours, or completion.

Day 1 — VFS Objects, Names, and Ownership

Target time: approximately 3 hours

> A file descriptor is not a file, a path is not an inode, and an inode is not an open operation. Keeping those identities separate makes lifetime bugs visible.

1. Start from the evidence

Before reading an abstract filesystem diagram, run `challenges/day-001-vfs-objects.sh` from the repository root. Predict three facts: `/hello` and `/note` name two inodes; a successful open returns a number from 5 through 8; and the offset belongs to an open-file slot rather than to the inode. The wrapper builds the canonical Week 9 image, checks its normal marker and status, and displays the current structures and symbols.

Do not turn a successful marker into more evidence than it is. It proves that the selected initialization and user path completed under this build and QEMU configuration. It does not prove concurrent safety, reclaim, arbitrary path handling, or persistence.

2. Why kernels introduce a VFS vocabulary

Programs want operations such as open, read, and write. Storage objects vary: a ram-backed file, a disk filesystem file, a pipe, a terminal, and a device node do not store bytes the same way. A virtual filesystem layer gives kernel code a stable vocabulary while concrete implementations supply operations.

Four concepts are especially useful:

- A **path** is a name supplied for resolution. It is transient input, not the stored object.
- A **dentry** associates a name with an object in a directory context and can cache successful or failed lookup. ToyOS has no dentry structure or cache.
- An **inode** represents filesystem object identity and metadata. ToyOS's inode embeds its data and stores a whole absolute path, size, and writable flag.
- A **file** or open-file description represents one open instance. It carries the current offset and points at an inode.
- A **file descriptor** is a small process-visible index that selects an open-file description.

ToyOS implements only a narrow shape of that model. The names are still worth using because they expose why the offset is not placed in `struct inode`. If two independent opens shared an inode-level offset, one reader would move the other reader's position. In ToyOS, each successful `vfs_open` selects a free `struct file`, stores the inode pointer, resets that slot's offset to zero, and returns `5 + slot`.

3. Build an ownership ledger

The two arrays in `vfs.c` have static storage duration. They are kernel-global and exist for the life of the kernel. `vfs_init` assigns two pointers to string literals and initializes inode contents. No allocation or destruction occurs.

An ownership ledger for this implementation is:

```
object                owner/lifetime                mutable state
path argument         syscall stack temporarily     copied characters
inode array           VFS global / kernel lifetime  data, size, writable
path string literals  kernel image / kernel lifetime immutable bytes
file array            VFS global / kernel lifetime  inode pointer, offset, open
returned descriptor   user-visible value            no object ownership itself
```

The descriptor does not own the inode. It encodes an array position. The file slot does not own dynamically allocated memory either; it borrows a pointer to a static inode. These facts make the current code simple, but they would be unsafe assumptions if inodes could be removed or mounts could disappear.

In a fuller kernel, ownership usually involves reference counts. A process descriptor table refers to an open-file description; duplicated descriptors may share that description and therefore its offset; the open file refers to an inode; dentries and inodes can outlive one pathname lookup because caches and active users retain them. Unlink removes a name but need not destroy an object that remains open. ToyOS has none of those transitions, so do not infer them from its type names.

4. Path resolution is policy, not string equality

ToyOS's lookup loops over two inodes and compares the supplied string to each stored absolute path. That is bounded only indirectly by the syscall's 48-byte path-copy buffer. There are no directories, components, `.` or `..`, mount points, symbolic links, current working directory, root confinement, permissions, or rename races.

A real path walk repeatedly resolves components under a starting directory. It must define locking and lifetime rules while names can change. A dentry cache can preserve positive results and negative results such as “this name was absent,” but cache validity is a kernel-policy question. The architecture does not decide whether symbolic links are followed or whether a mount can be crossed.

This distinction illustrates a theme for the week:

```
architecture/device contract: how bytes and privilege transitions work
kernel policy: what names mean and which operation is permitted
C lifetime: whether every pointer still names a live object
```

5. Inspect invariants before debugging symptoms

For a wrong read result, avoid immediately blaming user copying. Check the earliest relevant invariant:

1. Is the path NUL-terminated within the syscall's 48-byte capacity?
2. Did lookup select the intended inode?

3. Did open reserve a free file slot and set offset zero?
4. Does the descriptor decode back to that slot?
5. Is `file->inode` still valid?
6. Is `offset <= inode->size` before subtracting?
7. Did the operation move only that file slot's offset?

Current initialization makes `offset <= size` true for the demonstrated paths. The subtraction in `vfs_read` relies on that invariant. If future seek support allowed offsets beyond EOF, unsigned `size - offset` would underflow unless the read path first checked the relationship.

6. Compare the object model

xv6 separates descriptors, `struct file`, and filesystem inodes. Its open files are reference counted and can select inode-backed, pipe, or device operations. Its filesystem has directories, a buffer cache, logging, and a disk layout. Verify names in the revision used rather than copying line numbers from memory.

Linux has a much richer VFS: a task's descriptor table points at `struct file` objects; files refer to dentries and inodes; superblocks represent mounted filesystem instances; operation tables dispatch to implementations. Names, mount namespaces, credentials, caches, writeback, and concurrent lifetime all add invariants. The conceptual correspondence is useful, but ToyOS structures are not miniature drop-in Linux structures.

7. Explain the observed path

After running the wrapper, write a causal account of one `/hello` open. Name the source of the path bytes, the temporary kernel destination, the lookup comparison, the selected static inode, the selected static file slot, the offset initialization, the descriptor arithmetic, and the evidence that user execution later completed.

Then state the limit: no close means an open slot is never released. Repeating successful opens eventually reaches `-24`. That is a lifetime policy, not an x86 rule and not a ramfs storage limitation.

Practical challenge

Purpose: identify each Week 9 VFS object, its owner, and its lifetime.

Prediction: static inodes outlive every open, each open slot gets its own offset, and the checked image reaches `WEEK9: PASS` with host status 33.

Evidence: the wrapper prints current definitions and symbols, runs the canonical checked image, and records artifact identity.

Limits: no dentry object, close, reclamation, concurrency, permissions, directories, mount, or persistence is exercised.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Vfs Objects

Format: shell lab

What you will do: Audit Week 9 VFS object identity, ownership, and open-file offsets.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 09-filesystems-and-virtual-devices/challenges/day-001-vfs-objects.sh
```

- View lab source on GitHub

Day 2 — Tiny ramfs Lookup, Open, Read, and Write

Target time: approximately 3 hours

> Trace state changes before assigning filesystem names to them. In this ramfs, persistence means only “until this VM stops.”

1. Run the vertical slice first

Use `challenges/day-002-ramfs-io.sh` from the repository root. Before running it, predict the state transitions for `/hello`: lookup chooses inode 0, open chooses a free slot and offset 0, a read requests 64 bytes but returns only the file size, and EOF would return 0 after the offset reaches the size. Predict also that `/note` accepts the user write while `/hello` does not.

The wrapper checks the normal Week 9 image and the missing-path rejection profile. The rejection profile calls `vfs_open("/missing")` in kernel setup, asserts return `-2`, emits an exact marker, and exits with guest value `0x17`, which the harness observes as host status 47. It does not enter ring 3 and is not a user pathname test.

2. Initialization constructs the whole filesystem

`vfs_init` initializes two elements of a static inode array:

```
/hello  read-only  "USER: ramfs open/read through fd\n"
/note   writable   empty, then probed with "vfs\0"
```

`set_contents` copies until NUL or 256 bytes. It records the count of copied bytes and does not copy the terminator. The call with `"vfs"` passes a source string to `set_contents` only during setup? No: initialization instead opens `/note` and calls `vfs_write` with `sizeof(probe)`, so four bytes are written: `'v'`, `'f'`, `'s'`, and `'\0'`. File data may contain NUL; `size`, not a C string terminator, defines readable content.

This detail is a useful C lesson. Files are byte sequences. Treating file data as a C string would conflate a stored zero byte with end of file and could read beyond a valid range when no zero exists.

3. Lookup is exact and linear

`vfs_open` starts with a null inode pointer, walks the two-element array, and uses `strings_equal`. A match selects the address of that static inode. If none matches, it returns `-2`, corresponding conceptually to `ENOENT`.

This is whole-string equality, not a component walk. `/hello/`, `hello`, and `/HELLO` do not match `/hello`. There is no normalization. The loop is safe for the stored literals and for a properly NUL-terminated caller string. The user syscall path establishes that termination with a bounded byte-at-a-time copy; direct kernel callers are responsible for the same precondition.

After lookup, another linear scan finds the first `open == false` file slot. The slot borrows the inode pointer, resets offset to zero, sets `open`, and returns `VFS_FILE_BASE + i`. If all four slots are open, the function returns `-24`, conceptually `EMFILE`. Because there is no close, every successful open consumes one slot for the rest of the boot.

Initialization itself consumes one slot by opening `/note`; the user program then opens `/hello` and `/note`, leaving one free slot. This accounting matters when predicting capacity. “Four slots” does not mean the user begins with four available slots.

4. Read is bounded by request and EOF

`file_for` first validates the descriptor range `[5, 9)`, indexes the static array, and returns the slot only if `open` is true. `vfs_read` rejects a bad slot or null destination with `-9`.

For a valid slot:

```
available = inode->size - file->offset
count = min(requested length, available)
copy count bytes from inode data at offset
offset += count
return count
```

At EOF, `available` and `count` are zero. Returning zero distinguishes EOF from an error. A short read is not automatically a failure; it can mean that the request reached the current file size.

The code depends on the invariant `offset <= size`. Opens establish offset 0. Reads never advance beyond size. Writes either extend size to the new offset or remain within it. No seek exists, so the invariant holds on implemented paths. When extending the design, make this precondition explicit before the unsigned subtraction.

5. Write combines permission, capacity, and position

`vfs_write` returns `-9` when the file slot is invalid, the source pointer is null, or the inode is not writable. Combining those conditions simplifies ToyOS but loses diagnostic precision: a read-only file would conventionally produce a permission- or descriptor-mode-related error rather than “bad descriptor.”

Capacity is checked without overflow:

```
length > RAMFS_CAPACITY - offset
```

Given the offset invariant, subtraction is safe and avoids computing `offset + length` before confirming it fits. A failing write returns `-28` without copying a prefix and without moving the offset. A successful write copies all requested bytes, advances the open-file offset, and extends inode size if necessary. This all-or-error policy is a ToyOS choice; real writes can be partial.

Because each open has a separate offset but points at shared inode data, two opens can overwrite the same bytes independently. ToyOS has one process and no concurrency, so it avoids races accidentally rather than through locking.

6. Debug by state transition

When a file result is surprising, record a before/after table:

```
path | inode index | writable | slot | fd | old offset
request length | available/capacity | result | new offset | new size
```

Check the first disagreement with your prediction. If lookup selected no inode, inspecting copy-out is premature. If the descriptor is wrong, do not debug inode capacity yet. If return count is correct but user output is wrong, then inspect `copy_to_user`, the user buffer address, and the following serial write.

The controlled missing-path profile proves only one negative branch. It does not test slot exhaustion, read-only writes, capacity overflow, null buffers, EOF, repeated opens, or data persistence across reboot. List those as future tests instead of implying coverage.

7. Ramfs is not a disk filesystem

Every inode and data byte lives in kernel static memory. There is no block device, sector address, buffer head, bio, request queue, page cache, dirty tracking, writeback, superblock, free-space allocator, crash consistency, or journal. Resetting QEMU discards all changes.

That does not make the exercise fake. It isolates naming, object identity, open-file position, bounds, and user/kernel copying before adding storage failure and persistence. The right claim is “a tiny in-memory filesystem path works,” not “ToyOS has a storage stack.”

Practical challenge

Purpose: account for every lookup and byte-range decision in ramfs.

Prediction: normal lookup/read/write completes; `/missing` returns `-2` and the checked rejection profile exits with host status 47.

Evidence: canonical source excerpts, exact symbols, normal QEMU output, and the dedicated filesystem profile.

Limits: the wrapper does not manufacture unimplemented close, seek, directories, persistence, partial writes, or concurrent access.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Ramfs Io

Format: shell lab

What you will do: Trace tiny ramfs lookup, open, offset, read, write, and missing-path behavior.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 09-filesystems-and-virtual-devices/challenges/day-002-ramfs-io.sh
```

- View lab source on GitHub

Day 3 — User File Descriptors, Syscalls, and Error Lifetimes

Target time: approximately 3 hours

> A safe syscall path validates two lifetimes: the user memory range during copying and the kernel object selected by the descriptor during use.

1. Observe the user program before the dispatcher

Run `challenges/day-003-user-fd.sh`. First predict the register-level path: `SYS_OPEN` places 7 in `RAX` and a user path pointer in `RDI`; the result is returned in `RAX`. `SYS_READ` uses 6 with descriptor in `RDI`, destination in `RSI`, and length in `RDX`. File writes reuse `SYS_WRITE` number 1: descriptor 1 selects serial, descriptor 4 selects the old pipe, and other descriptors are offered to VFS in Week 9.

The wrapper checks the exact image and disassembles the user entry. Source shows intent; disassembly establishes what the selected ELF contains; QEMU output establishes one executed path. Keep these evidence classes distinct.

2. Open copies a string across the trust boundary

The syscall dispatcher cannot pass a user pointer directly to `vfs_open`. User virtual addresses belong to the active user address space and may be unmapped, supervisor-only, or end at a page boundary. `copy_user_string` copies one byte at a time through `copy_from_user`, up to a 48-byte kernel array.

It returns true only after copying a NUL. An invalid translation produces false and the syscall returns `-14`. A valid 48-byte sequence without NUL also returns false and currently maps to the same `-14`, although conceptually it is a length/termination error rather than a bad address. The copied kernel array has automatic storage duration inside the `SYS_OPEN` case. It remains live during `vfs_open`, which compares immediately and does not retain the pointer. Retaining it in an inode would create a dangling pointer after dispatch returned.

Copying one byte at a time is simple and naturally checks page transitions, but it is inefficient. A production helper might copy chunks while preserving fault recovery and termination semantics. Performance does not justify skipping validation.

3. Descriptors select policy

ToyOS Week 9 has no per-process descriptor table. Instead, descriptor values have hard-coded meanings:

```
1       serial output
3       global pipe read end
4       global pipe write end
5..8    VFS file slots 0..3
```

The single process makes this convention usable for teaching, but it is not a general descriptor model. A full process descriptor table permits the same integer in different processes to select different open files, supports close, duplication, inheritance, flags, and controlled sharing.

The dispatcher first stages up to 256 bytes in its kernel stack buffer. For a file write it checks and copies user bytes before calling `vfs_write`. For a file read, it calls `vfs_read` into the kernel buffer and only then copies the returned count to user memory. This staging ensures ramfs never directly dereferences a user pointer.

4. Trace ownership through read and write

For a file read:

```
user owns mapped destination bytes
dispatcher owns temporary 256-byte stack buffer
file slot owns current offset
inode owns ramfs data and size
vfs_read copies inode → kernel buffer and advances offset
copy_to_user copies kernel buffer → validated user pages
```

There is a subtle policy issue: the offset advances before `copy_to_user`. If user copy-out fails, the syscall returns `-14`, but the file position has already moved. A production design must intentionally define and test this failure behavior, often by faulting in pages, locking state, or handling partial copies. ToyOS does not roll back the offset.

For a file write, user copy-in completes before the inode changes. If copy-in fails, VFS sees no operation. If VFS rejects capacity, no bytes or offsets change. These orderings are part of syscall policy, not an ISA requirement.

5. Classify errors by detection boundary

Current numeric results include:

- `-2`: VFS lookup found no matching inode.
- `-9`: descriptor/slot or VFS buffer validity failed; also used for read-only writes.
- `-14`: user copy or bounded string copy failed.
- `-22`: read length exceeded the dispatcher's 256-byte staging capacity.
- `-24`: no open-file slot remained.
- `-28`: write would exceed the 256-byte inode capacity.
- `-38`: syscall number was unknown.

Do not describe these as complete POSIX `errno` behavior. They are negative return conventions in this kernel. The assembly caller receives the two's complement value in `RAX`; there is no libc translating it into `errno`.

Debugging begins at the earliest detector. For `-14`, inspect the active CR3, the user virtual address and page walk, access permission, requested range, and string terminator. For `-2`, first prove the kernel copy succeeded, then inspect exact path bytes. For `-9`, decode the descriptor and file-slot open state. For `-28`, record offset, length, and remaining capacity.

6. Lifetime is incomplete without close

The static inode and file arrays never disappear, so pointers remain valid for kernel lifetime. But resources leak by design: no syscall clears `file.open`. The initialization probe consumes one slot, and the user path consumes two more. A future close must validate the descriptor, release exactly one reference, prevent use-after-close, and define behavior when descriptors share an open file.

Adding processes would require deciding whether descriptor state belongs to a process, how fork shares open-file descriptions, how exec preserves or closes entries, and what teardown releases. Locks would become necessary if another CPU or task could mutate slots and offsets concurrently. A raw pointer plus a Boolean is not enough for those lifetimes.

7. Compare syscall boundaries

xv6 copies pathname arguments into kernel memory and maps process descriptor indices to reference-counted `struct file` objects. Its `read` and `write` dispatch according to file type and coordinate inode operations through its filesystem and logging rules.

Linux separates syscall entry, descriptor lookup, `struct file` lifetime, VFS operations, filesystem implementation, and architecture-aware user-copy handling. Linux can return short counts, restart interrupted operations, apply credentials and flags, and coordinate concurrent close. Those mechanisms address concrete invariants absent from ToyOS.

The architecture difference is primarily at syscall and user-memory access. ToyOS uses x86 `int 0x80`, page-table U/S checks, and a software walk. xv6 on RISC-V uses `ecall` and supervisor page-table conventions. VFS concepts such as separate open offset and inode identity remain kernel-policy concepts.

Practical challenge

Purpose: follow user bytes, descriptor values, kernel buffers, offsets, and errors through the implemented syscall path.

Prediction: the canonical user opens `/hello`, reads its current size, writes that count to serial, opens `/note`, writes its message, and exits with the normal checked result.

Evidence: source and ELF instruction sequence, dispatcher/VFS contracts, artifact hashes, and QEMU markers.

Limits: no libc, process-local fd table, close, dup, fork inheritance, blocking, rollback after copy-out failure, or concurrent mutation exists.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — User Fd

Format: shell lab

What you will do: Trace Week 9 user open/read/write registers, copying, descriptors, and lifetimes.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 09-filesystems-and-virtual-devices/challenges/day-003-user-fd.sh
```

- View lab source on GitHub

Day 4 — File to Page Cache to Filesystem to Block: A Conceptual Boundary

Target time: approximately 3 hours

> ToyOS demonstrates file semantics over static RAM. It does not demonstrate a page cache, block layer, disk filesystem, writeback, or journaling.

1. Prove what is absent

Run `challenges/day-004-storage-boundary.sh`. Predict that it will find the ramfs arrays and direct byte-copy loops, while searches for block requests, dirty pages, journal transactions, sector I/O, and writeback find no Week 9 implementation. The wrapper also runs the canonical image so absence claims are tied to the same project that supplies the positive VFS evidence.

An absence argument should be scoped carefully. Searching selected source files is not a mathematical proof about every possible artifact. Here the project source map, build inputs, symbols, and direct implementation agree: Week 9 links `vfs.c` and `edu.c`; ramfs data is embedded in inodes; EDU is used only for MMIO liveness. No block storage path is claimed.

2. Build the conceptual read path

In a conventional cached filesystem, a buffered user read may follow this conceptual chain:

```
fd → open file and offset → inode/mapping → page-cache lookup
   → cache hit: copy cached bytes to user
   → cache miss: filesystem maps file offset to storage blocks
               → block I/O request → device driver → controller/media
               → completion → page becomes uptodate → copy to user
```

Each arrow has ownership and waiting rules. The open file owns a position. The inode identifies the file. Cached pages need references and state such as locked, uptodate, dirty, or under writeback. A block request owns a transfer description until completion. A driver owns queue and device state. User memory must remain safely accessible for the copy, but buffered I/O normally does not let a storage controller write arbitrary user pointers.

ToyOS collapses that chain:

```
fd → static file slot → static inode → embedded 256-byte array
```

There is no cache miss because there is no lower backing store. Calling the inode byte array a “page cache” would be false: it is the authoritative storage for this boot, not a cached copy of another object.

3. What a page cache contributes

A page cache stores file content in memory in page-sized units and coordinates reads, writes, mappings, reclaim, and writeback. It must answer questions that ToyOS can ignore:

- Is this page present and uptodate?
- Who waits while I/O fills it?
- Can reclaim remove it?
- If dirty, when and how is it written back?
- What happens when writeback fails?
- How do a memory mapping and `read` observe coherent bytes?
- What locking prevents duplicate fills or stale size observations?

The CPU page tables that map virtual memory are not the page cache. Both may deal in page-sized memory, but they solve different problems. Page tables translate addresses and enforce access permissions. The page cache represents cached file content and its I/O state. ToyOS's user copying walks page tables; that does not imply a file cache.

4. Filesystem mapping and metadata

A persistent filesystem must translate a logical file offset into storage locations while maintaining metadata: file size, allocation, directory entries, ownership, timestamps, and free space. Sparse regions may read as zero without allocated blocks. Extending a file may allocate blocks and must handle partial failure.

ToyOS's “mapping” is array indexing: `inode->data[file->offset + i]`. Its metadata consists of path pointer, size, and writable Boolean. It has no on-media format, superblock, allocation bitmap, directory records, checksum, or mount operation.

xv6 offers a useful next step because its filesystem has inodes, directories, block mapping, a buffer cache, and a write-ahead log over a virtio disk. Linux supports many filesystem implementations over generic VFS, page-cache, writeback, and block interfaces. Exact paths and APIs evolve; inspect the revision being discussed.

5. The block layer and driver boundary

A block layer accepts operations over logical sectors or blocks, schedules or merges work according to policy, and submits requests to a driver. The driver translates those requests into a device protocol, manages registers and descriptors, and reports completion.

The Week 9 EDU device is not used as a block device. ToyOS writes one liveness register and reads its inverted value. It neither transfers file data through EDU nor submits sector requests. EDU has educational DMA and interrupt facilities, but this project explicitly does not enable them.

If a future ToyOS storage driver used programmed I/O, the CPU might copy data through device registers while polling status. An interrupt-driven driver would submit work, retain request state, and complete later. DMA would require device-visible physical addresses, pinned buffers, ownership transitions, cache/coherency handling where applicable, and strict length validation. Those are new invariants, not automatic consequences of mapping a BAR.

6. Why journaling is a different layer

Journaling or write-ahead logging protects selected filesystem invariants across crashes by controlling the order in which metadata and sometimes data become durable. A typical transaction records intended changes in a log, persists enough information, commits, then installs or checkpoints changes. Recovery replays committed work and ignores incomplete work.

Memory barriers and MMIO ordering do not provide crash consistency. A CPU barrier can constrain ordering of operations as observed by processors or devices under an architecture's rules. It does not by itself guarantee that a storage device has committed data to durable media. Flush and force-unit access protocols, controller caches, filesystem transaction rules, and error handling matter.

ToyOS has no journaling because it has no persistent filesystem or block write path. Its static RAM changes vanish at reset. Describing the simple all-or-error ramfs write as “transactional” would confuse in-memory control flow with durable atomicity.

7. Design the next slice without pretending it exists

A defensible next teaching slice might add a tiny read-only block-backed filesystem:

1. identify a suitable virtual block device;
2. define a synchronous, polled block-read contract;
3. allocate a kernel-owned aligned buffer;
4. validate device-visible addresses and lengths;
5. read and verify one fixed superblock;
6. map one file's logical block;
7. fill a simple cache entry;
8. expose it through existing open/read semantics;
9. inject timeout, malformed metadata, and out-of-range block tests.

Only after that works should claims include block I/O. Dirty data, writeback, crash recovery, IRQ completion, and DMA each deserve separate milestones and negative tests.

Practical challenge

Purpose: distinguish the implemented ramfs from the conceptual persistent storage stack.

Prediction: evidence shows direct inode-array copies and no real page cache, block request, journaling, or file-data device operation.

Evidence: canonical build inputs, source paths, symbol inventory, and a checked normal run.

Limits: text and symbol inspection cannot validate an absent future design; the conceptual path is a design model, not a ToyOS execution trace.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Storage Boundary

Format: shell lab

What you will do: Establish the implemented ramfs boundary versus a conceptual storage stack.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 09-filesystems-and-virtual-devices/challenges/day-004-storage-boundary.sh
```

- View lab source on GitHub

Day 5 — PCI Configuration Enumeration, BARs, and QEMU EDU

Target time: approximately 3 hours

> Discovery reads a standardized configuration interface. A matched ID tells the kernel what device may be present; it does not prove a usable register mapping or a completed operation.

1. Predict the discovery transaction

Run `challenges/day-005-pci-edu.sh`. Before execution, predict that ToyOS scans device numbers 0 through 31 on bus 0, function 0; finds the combined vendor/device value `0x11e81234`; reads BAR0 at offset `0x10`; requires a memory BAR and nonzero base; maps one page; then checks EDU's ID register.

The wrapper invokes QEMU with `-device edu` through the project's canonical test target. Omitting that option is not a valid negative test under the current code because `edu_init` asserts that discovery succeeds and would panic rather than emit a dedicated expected marker.

2. Configuration mechanism #1 on x86

ToyOS uses legacy PCI configuration mechanism #1:

```
CONFIG_ADDRESS port 0xcf8
CONFIG_DATA    port 0xcfc
```

It forms a 32-bit address with enable bit 31, device in bits 15:11, and the dword-aligned register offset in bits 7:2. Bus and function fields remain zero. It writes the address with `outl`, then reads the data with `inl`.

Conceptually, the full address is:

```
enable | bus<<16 | device<<11 | function<<8 | (offset & 0xfc)
```

Because ToyOS scans only bus 0/function 0, it cannot discover devices behind bridges, on other buses, or in nonzero functions. It also does not inspect header type to determine multifunction behavior. This is sufficient for the QEMU machine arrangement selected by the test, not general PCI enumeration.

The inline assembly is x86-specific. I/O ports and `inl/outl` are not portable C and do not exist as the PCI access mechanism on many AArch64 or RISC-V platforms. Those systems commonly expose an ECAM or host-controller MMIO window described by ACPI or device tree. The generic PCI concepts—BDF, configuration space, IDs, BARs—remain; the access mechanism and discovery source differ.

3. Interpret vendor and device IDs precisely

Reading offset zero returns vendor ID in the low 16 bits and device ID in the high 16 bits. On little-endian x86, QEMU EDU vendor `0x1234` and device `0x11e8` appear in a 32-bit read as `0x11e81234`.

ToyOS compares that complete value. It does not inspect class code, revision, subsystem IDs, capabilities, or command register. A match identifies the device model expected by this lesson, but robust binding normally combines tables, class information, firmware policy, and driver-specific checks.

No claim should be made that the scan “configures PCI.” It reads enough configuration state to locate one device and one BAR. It does not assign bus numbers or resources, enable bus mastering, negotiate MSI/MSI-X, or preserve firmware topology.

4. Decode BAR0 without overgeneralizing

At configuration offset `0x10`, BAR0 describes an I/O or memory resource. Bit 0 distinguishes I/O space from memory space. ToyOS asserts bit 0 is clear and masks the low four flag bits:

```
physical = bar & ~0xf
```

It also requires a nonzero result. This is adequate for the selected 32-bit memory BAR assigned by QEMU. General BAR handling must inspect memory type, possibly combine adjacent low/high dwords for a 64-bit BAR, account for prefetchability, determine resource size, and validate assignment. Drivers must not casually write all ones to a live BAR to size it after the platform has configured resources without appropriate coordination.

ToyOS maps exactly one 4 KiB page at virtual address `0x60000000` with `PTE_WRITE`. It assumes BAR0 is page aligned and that the used registers fit in that page. The page tables currently do not express a dedicated uncacheable/device memory type. On x86, platform MTRR/PAT state and the QEMU environment make this selected test work, but a production mapping API must choose suitable memory attributes explicitly.

5. Separate four validation stages

A useful driver bring-up report records:

1. **Discovery:** which BDF produced which vendor/device dword?
2. **Resource decoding:** what raw BAR was read, what flags were removed, and what physical range is claimed?
3. **Mapping:** what virtual range and page-table attributes expose it?
4. **Device identity:** what value did the mapped ID register return?

Failure at one stage should not be debugged as another. If no ID matches, inspect QEMU arguments and config transactions. If BAR bit 0 indicates I/O, do not debug page tables. If the mapping faults, inspect the PTE and physical base before questioning EDU semantics. If the ID is wrong but reads are stable, verify register width, endianness, offset, and device revision.

Artifact identity includes more than the ELF. The virtual hardware command line is part of the experiment. A report should record QEMU version and the presence of `-device edu`.

6. Architecture and policy boundaries

The PCI specification assigns configuration fields and BAR meaning. x86 defines I/O instructions and memory-order behavior. QEMU EDU defines its register protocol. ToyOS policy decides to scan only 32 slots, bind the first exact ID, choose virtual address `0x60000000`, map one page writable, and panic on mismatch.

On AArch64, page tables must map device registers with a device memory type; ordinary cacheable memory is wrong. On RISC-V, platform descriptions and physical memory attributes matter, and ordering may require `fence`. These differences are relevant when porting `edu.c`, but they do not change VFS descriptor policy.

7. Define what discovery does not establish

Successful enumeration does not establish interrupt routing, DMA safety, concurrent access, reset handling, hotplug, power management, error recovery, or performance. It also does not establish that every register offset is valid. Day 6 examines checked access and ordering.

The narrow Week 9 claim is still valuable: the canonical kernel issues real x86 PCI configuration transactions against QEMU's EDU model, decodes its assigned BAR, maps that physical resource, and validates an EDU-defined ID. That is stronger than a mock structure and weaker than a complete driver.

Practical challenge

Purpose: reconstruct PCI config addresses and validate the chain from ID match through BAR mapping.

Prediction: one bus-0/function-0 slot matches `1234:11e8`, BAR0 is a nonzero memory BAR, and the normal profile reaches the EDU pass marker.

Evidence: exact source, ELF symbols and disassembly, QEMU version and command contract, plus checked serial/status output.

Limits: no bridge walk, multifunction scan, ECAM, resource sizing, 64-bit BAR handling, interrupt setup, DMA enablement, or hotplug is tested.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Pci Edu

Format: shell lab

What you will do: Audit x86 PCI config-space enumeration, BAR0 decoding, and EDU identification.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 09-filesystems-and-virtual-devices/challenges/day-005-pci-edu.sh
```

- View lab source on GitHub

Day 6 — MMIO Register Access, Ordering, and EDU Boundaries

Target time: approximately 3 hours

> `volatile` constrains compiler treatment of an access. It is not a complete device-ordering API, a lock, an interrupt protocol, or a DMA ownership model.

1. Exercise the one implemented operation

Run `challenges/day-006-mmio-edu.sh`. Predict that the kernel reads EDU ID `0x010000ed`, writes `0x12345678` to liveness offset `0x04`, and reads back the bitwise inverse. Predict also that the device rejection profile calls `edu_checked_write(3, 0)`, rejects the unaligned and unauthorized offset before MMIO, emits its marker, and exits with host status 49.

The rejection profile first completes normal EDU initialization. It therefore tests the helper's software validation after discovery and liveness have succeeded. It does not provoke a hardware bus fault or ask EDU to reject an operation.

2. Understand the C expression and the bus operation

`edu_mmio` has type `volatile uint32_t *`. Indexing by `offset / sizeof(uint32_t)` forms a 32-bit access at the selected register. The `volatile` qualification tells the compiler that accesses are observable: it should emit them rather than remove, merge, or satisfy them solely from a register under normal compiler rules.

That property is necessary for this direct style but insufficient as a general driver interface:

- It does not make a multi-step protocol atomic.
- It does not serialize two CPUs.
- It does not define device-endian conversion.
- It does not select page-table memory attributes.
- It does not necessarily provide every ordering relation needed around ordinary memory and MMIO on every architecture.
- It does not flush posted writes merely because C execution reached the next statement.

Production kernels wrap register access in architecture-aware helpers such as read/write APIs. Those helpers encode width, endianness, barriers, and platform quirks more reliably than scattered volatile pointer expressions.

3. Validate registers before touching them

`edu_checked_write` accepts a write only when:

1. the base pointer has been installed;
2. offset is exactly the liveness register `0x04`;
3. offset is four-byte aligned;

4. offset lies within the mapped page.

The exact-offset condition is strongest here; it prevents the helper from writing the read-only ID register or any unimplemented EDU register. The alignment and range checks document general MMIO hazards even though equality already implies them for the one allowed offset.

Order the checks so no pointer arithmetic or device access occurs first. Invalid offsets are untrusted driver inputs even when only kernel code calls the helper today. If later exposed indirectly to users, validation remains in kernel policy.

The helper returns false rather than panicking. `edu_init` chooses to assert success for its known liveness write. This separates a reusable checked operation from boot-time policy.

4. Reason about ordering by architecture

x86 has a relatively strong memory model, and port I/O instructions plus uncacheable MMIO have architecture-defined ordering properties. Compiler barriers and proper MMIO mapping attributes still matter. A read from a device can also be used according to the device/platform contract to drain a posted write, but an arbitrary read is not universally a safe flush.

AArch64 is more weakly ordered. MMIO should be mapped as device memory with the correct shareability and access attributes. Kernel accessors may include ordering, while explicit `dmb` or `dsb` is required at protocol-defined boundaries. The distinction between ordering accesses and waiting for effects to complete matters.

RISC-V likewise requires attention to I/O versus memory ordering and may use `fence` with appropriate predecessor/successor sets. Platform memory attributes and coherent interconnect behavior are part of the contract.

Do not cargo-cult a barrier after every access. Start from the protocol: which prior ordinary-memory writes must be visible before a doorbell? Which status read must occur after a command? Which CPU may race? Then choose the kernel accessor and barrier that creates those relations on the target ISA.

ToyOS performs a simple write followed by a read of the same liveness register. The QEMU EDU model returns the inverse, providing a deterministic functional check. The project does not present this as a portable ordering library.

5. The EDU operation and evidence

The sequence is:

```
mapped base established
ID[0x00] read as 0x010000ed
checked write of 0x12345678 to LIVENESS[0x04]
LIVENESS[0x04] read as ~0x12345678
serial pass marker emitted
```

The readback demonstrates that the selected mapping reaches a device implementing the expected EDU liveness semantics in this QEMU run. It helps catch a wrong BAR, wrong virtual mapping, wrong offset, or absent device. It does not prove behavior on physical hardware, stress ordering, or validate other EDU registers.

When the value is wrong, record raw values instead of immediately adding delays. Verify device presence, raw BAR, physical mask, PTE, memory type, virtual address, access width, alignment, register offset, and QEMU version. For a fault, also record saved RIP, fault address, and page-fault error bits.

6. IRQ is an explicit future boundary

EDU can generate interrupts, but Week 9 does not configure or claim them. Adding IRQ operation requires at least:

- a known routing mechanism and interrupt controller path;
- interrupt enable/acknowledge semantics from EDU's specification;
- a handler registered before enabling the source;
- shared state whose lifetime extends until completion;
- synchronization between process and interrupt context;
- timeout and spurious/unexpected interrupt behavior;
- masking and teardown order that prevent use-after-free.

This minimal kernel uses a legacy PIC for earlier timer work. PCI INTx routing under the selected virtual machine is not established here. MSI/MSI-X setup is also absent. Polling one register is not evidence of interrupt delivery.

7. DMA is a separate future boundary

EDU also supports DMA, but Week 9 does not enable bus mastering or submit a DMA command. A correct DMA path must distinguish virtual from physical/device addresses, ensure the device can address the buffer, pin ownership until completion, validate size and alignment, handle cache coherency, and prevent the device from accessing unrelated memory. An IOMMU can improve isolation but requires its own mapping and lifetime model.

ToyOS's frame allocator and identity-access assumptions do not provide a complete DMA API. Passing a C pointer to a device register would be an unsafe fiction. The right next claim remains: “IRQ and DMA are not implemented.”

Practical challenge

Purpose: connect a checked C register access to device-defined behavior while separating compiler, CPU, mapping, and device contracts.

Prediction: liveness returns the inverse; offset 3 is rejected in software and the expected profile reaches host status 49.

Evidence: helper and mapping source, exact instructions, normal checked operation, and checked invalid-register profile.

Limits: no portable MMIO accessor layer, explicit cache-attribute API, concurrency, physical-device validation, IRQ completion, or DMA exists.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Mmio Edu

Format: shell lab

What you will do: Verify checked EDU MMIO liveness and invalid-register software rejection.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 09-filesystems-and-virtual-devices/challenges/day-006-mmio-edu.sh
```

- View lab source on GitHub

Day 7 — Integrated Filesystem and Device Fault/Debug Report

Target time: approximately 3 hours

> A useful driver report ties every conclusion to artifact identity, mutable ownership, a detection boundary, and an explicit non-claim.

1. Generate evidence before writing conclusions

Run `challenges/day-007-week9-report.sh`. Predict normal/filesystem/device host statuses 33, 47, and 49 with exact markers. The wrapper creates a temporary report, runs the full canonical Week 9 regression, hashes the relevant ELFs, inspects symbols and headers, and extracts the implemented contracts.

Generated output is not the explanation. Add your own causal trace, including which state changes before each result and which absent feature prevents a broader conclusion. Never present suggested GDB commands as observations.

2. Trace user open/read/write into ramfs

Begin at `_user_start` in the exact Week 9 user ELF:

```
RAX=7, RDI=&"/hello" → int 0x80
gate/stub → syscall_dispatch
copy_user_string → page-wise checked copy into 48-byte kernel array
vfs_open → exact lookup → static inode → free file slot at offset 0
fd returned in saved RAX → iretq

RAX=6, RDI=fd, RSI=&io_buffer, RDX=64 → int 0x80
vfs_read → inode bytes into dispatcher stack buffer; file offset advances
copy_to_user → checked user destination
count returned → user invokes write fd 1 → COM1 output
```

Then trace `/note`. Initialization has already opened it and written four probe bytes. The user opens it independently, obtaining another file slot with offset zero, and writes `"user-fd-write"` over the beginning. The inode is shared; the offsets are not.

For every step record:

- privilege level and active address space;
- source and destination byte owner;
- path, inode, file slot, and descriptor identity;
- old/new offset and size;
- return value and earliest error detector;
- source, ELF, or runtime evidence type.

This catches a common compressed but misleading statement: “the user reads the inode.” The user never receives an inode pointer. It supplies a descriptor and user buffer; kernel code resolves the slot and stages bytes through a kernel stack buffer.

3. Trace PCI discovery into MMIO

The independent device chain is:

```
outl 0xcf8 with enabled bus0/deviceN/function0/offset0 address
inl 0xcfc → compare 0x11e81234
read BAR0 at offset 0x10 → reject I/O BAR → mask low flags
map physical BAR page at virtual 0x60000000 writable
volatile 32-bit ID read → compare 0x010000ed
checked aligned write at 0x04 → read inverse → pass marker
```

Record the QEMU version and `-device edu`; the virtual machine configuration is part of artifact identity. Distinguish the PCI standard, x86 config-port mechanism, QEMU EDU register contract, page-table mapping mechanism, and ToyOS policy.

The two Week 9 slices meet only in boot orchestration and the final successful user exit. EDU does not back ramfs. Ramfs does not feed a block request to EDU. An integrated report must not draw a nonexistent data arrow between them.

4. Interpret the controlled rejection profiles

Filesystem profile (`EXPECT_FAULT=8`). During `vfs_init`, after the normal `/note` probe, kernel code calls `vfs_open("/missing")`. Exact lookup returns `-2`; the assertion verifies that result; the profile emits `EXPECTED-FAULT: filesystem PASS` and writes guest exit value `0x17`, observed as host status 47. It does not test user copying or descriptor dispatch.

Device profile (`EXPECT_FAULT=9`). `edu_init` completes discovery, mapping, ID, and liveness. It then calls `edu_checked_write(3, 0)`. The helper rejects the request because offset 3 is not the only authorized liveness offset and is unaligned. The profile emits `EXPECTED-FAULT: device PASS` and guest value `0x18` appears as status 49. It does not issue invalid MMIO.

An unexpected page fault belongs to neither selected profile. Gather CR2, error bits, saved RIP/CS/RSP, active CR3, the relevant PTE chain, raw BAR, and exact ELF. If the fault occurs before mapping, debug setup; if the access completes with a wrong value, debug address/width/device semantics.

5. Use GDB as an evidence instrument

The provided Week 9 command file loads `build/week9/toyos.elf`, connects to `:1234`, and sets breakpoints at `kernel_main`, `vfs_init`, `edu_init`, and `syscall_dispatch`. Before an interactive session, predict:

- the matched PCI device number and raw BAR;
- `edu_mmio` before and after mapping;
- ID and liveness values;
- path bytes in the dispatcher array;
- selected inode and file slot;
- descriptor and offset before/after read;

- saved syscall registers and return `RAX`.

If symbols disagree with execution, first verify the paused VM uses that ELF and profile. If a descriptor appears invalid, derive its slot arithmetically before inspecting random memory. If an MMIO read is suspicious, do not repeat writes blindly; a real device register may have side effects.

6. Compare ToyOS, xv6, and Linux

ToyOS Week 9 uses static arrays and synchronous direct copies. It has one process, two inodes, four global slots, fixed absolute paths, no close, and no persistent storage. Device work is one polled MMIO operation after minimal PCI discovery.

xv6-riscv has per-process descriptor tables, reference-counted open files, inodes and directories, a buffer cache, block allocation, a write-ahead log, and a virtio disk path. Its device discovery and interrupt mechanisms are RISC-V platform-specific rather than x86 config-port based. It is still educational, but its storage lifetime and crash model are materially richer.

Linux separates descriptor tables, open files, dentries, inodes, superblocks, filesystem implementations, page cache/writeback, block I/O, and many driver models. PCI core code enumerates topology and resources; managed mapping and MMIO accessors encode platform requirements; interrupt and DMA APIs establish lifetime and ordering contracts. Linux also deals with hotplug, power management, IOMMUs, concurrency, and recovery.

The comparison is useful only when tied to an invariant. Reference counting prevents premature destruction. Dentry/inode separation handles naming versus identity. A page cache coordinates file data in memory. Journaling protects selected metadata invariants across crashes. DMA APIs protect ownership and addressability. Interrupt APIs coordinate asynchronous completion.

7. Preview a defensible driver path

A next EDU driver milestone could remain polled:

1. define typed register offsets and read/write helpers;
2. introduce explicit MMIO mapping attributes;
3. read and validate device state;
4. submit one bounded operation;
5. poll with a finite timeout;
6. report device error status;
7. inject timeout, invalid length, and wrong-state tests;
8. serialize access if more than one caller exists.

An IRQ milestone should follow only after routing and shared-state lifetime are defined. A DMA milestone should follow only after buffer physical addressing, pinning, ownership, bus-master enablement, coherency, and bounds are explicit. Connecting file I/O to a device would additionally require a block contract, cache policy, and filesystem mapping. None is implied by the current liveness test.

8. State conclusions narrowly

Week 9 demonstrates:

- separation of static inode identity and static per-open offset state;
- exact bounded lookup of two absolute paths;
- bounded ramfs reads and writes through user descriptors;
- checked user path and data copying;
- x86 PCI bus-0/function-0 configuration reads;
- exact QEMU EDU identification and BAR0 mapping;
- checked 32-bit liveness MMIO behavior;
- controlled missing-path and invalid-register rejection.

It does not demonstrate close/reclaim, directories, permissions, mount semantics, persistence, page cache, block I/O, journaling, concurrent access, portable MMIO mapping, interrupts, DMA, physical hardware, or a filesystem backed by EDU.

Practical challenge

Purpose: produce one reproducible report connecting user file operations, VFS/ramfs state, PCI/MMIO bring-up, fault boundaries, and comparisons.

Prediction: checked outputs are 33/47/49; the filesystem fault ends in kernel initialization, while the device fault rejects offset 3 before MMIO.

Evidence: tool and artifact identity, full Week 9 regression, exact symbols/source, integrated traces, and explicit limits.

Limits: the wrapper cannot substitute for an actually captured GDB session and makes no page-cache, block, IRQ, DMA, or persistence claim.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Week9 Report

Format: shell lab

What you will do: Generate a temporary integrated Week 9 filesystem/device evidence report.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 09-filesystems-and-virtual-devices/challenges/day-007-week9-report.sh
```

- View lab source on GitHub